Botswana Accountancy College
School of Computing and Information Systems

Peer-to-Peer Ride-Sharing Mobile Application for Urban Commuting in Botswana

by Boago Nkhii, Jaden Willams, Kaene Serumola, Katlego Baitirile

October 19, 2026

Contents

1 Introduction **1**
- 1.1 Project Overview . . . 1
- 1.2 Requirements . . . 2

2 Background Literature **3**
- 2.1 Problem Domain: Urban Transportation and Ride-Sharing . . . 3
- 2.2 Existing Ride-Sharing and Ride-Hailing Systems . . . 4
- 2.3 Ride-Matching and Compatibility . . . 8
- 2.4 Geolocation and Route Matching . . . 11
- 2.5 Factors Affecting Ride Compatibility . . . 13

3 System Analysis and Design **17**
- 3.1 Requirement Analysis . . . 17
  - 3.1.1 Major Use Case Scenarios . . . 17
  - 3.1.2 Identification of Classes from Use Cases . . . 19
- 3.2 Overview of Design . . . 19
  - 3.2.1 System Architecture . . . 19
  - 3.2.2 Architectural Components . . . 20
  - 3.2.3 Matching Process . . . 22
- 3.3 Entity Relationship Diagram (ERD) . . . 23
- 3.4 Class Diagrams . . . 24
  - 3.4.1 Entity Classes . . . 24
  - 3.4.2 Service Classes . . . 26
  - 3.4.3 Requirements Traceability . . . 28

Bibliography **31**

A Requirements Catalogue **33**
- A.1 Functional Requirements . . . 33

List of Figures

3.1 Use Case Diagram for the ride-sharing system 18
3.2 High-level architecture of the ride-sharing system 20
3.3 Flow chart of the ride-matching process 22
3.4 Entity Relationship Diagram (ERD) of the database 23
3.5 Entity classes of the system 24
3.6 Service classes of the system 26

List of Tables

2.1 Comparison of existing ride-sharing and ride-hailing systems 7

3.1 Operations of the entity classes 25
3.2 Operations of `RideService` and `RequestService` 27
3.3 Operations of `MatchingService` 28
3.4 Traceability of requirements to components and classes 28

Chapter 1

Introduction

1.1 Project Overview

The fundamental issue addressed by this project is the limited availability of convenient, localised, and affordable peer-to-peer ride-sharing services for everyday commuters in Botswana. Urban mobility in cities such as Gaborone can present financial and logistical challenges for commuters who depend on private vehicles or public transport. Existing informal transport systems provide important mobility services in areas where formal transport provision may be limited, but they can also experience challenges relating to coordination, service coverage, and reliability (Cervero & Golub 2007).

Local reporting has also highlighted challenges associated with public transport routes in Botswana. For example, a 2013 report on combi operations in Francistown described concerns regarding route extensions and the difficulties experienced by operators when routes did not adequately serve particular areas (Kebareng 2013). Such issues illustrate how the availability and configuration of transport routes can affect access to destinations and the operation of local transport services.

Ride-sharing provides an alternative approach in which people travelling along similar routes can share available vehicle capacity. Research on ride-sharing has identified potential benefits including reduced travel costs, improved utilisation of available vehicle capacity, and reductions in the number of vehicles required for some journeys (Agatz et al. 2012, Furuhata et al. 2013). However, effective ride-sharing requires mechanisms for identifying compatible drivers and passengers based on factors such as route, departure time, available seating, and acceptable additional travel.

To address this opportunity, a peer-to-peer (P2P), geolocation-based ride-sharing mobile application is being developed for urban commuting in Botswana. The system is intended to provide a localised platform through which private vehicle owners with available seats can connect with passengers travelling along compatible routes, allowing commuting

costs to be shared. The application is being developed using Expo and React Native for cross-platform mobile development, with Supabase providing backend services including authentication and database functionality.

1.2 Requirements

The system's requirements are summarised at a high level below and are divided into primary and secondary requirements. The complete requirement catalogue, including the full functional and non-functional specifications, is provided in Appendix A.

Primary Requirements

- Users shall be able to register and authenticate before accessing protected functionality.
- Drivers shall be able to post a ride offer specifying their origin, destination, departure time and available seats.
- Passengers shall be able to submit a ride request specifying their origin, destination and travel information.
- The system shall match drivers and passengers based on route, location, departure-time and seating compatibility.
- Drivers shall be able to accept or decline incoming ride requests.
- The system shall store and retrieve user profiles, ride offers, ride requests and ride histories.

Secondary Requirements

- Users shall be able to view the current status of a ride or request.
- Users shall be able to view their history of previous rides and requests.
- Users shall be able to view and select locations using an interactive map.

Chapter 2

Background Literature

2.1 Problem Domain: Urban Transportation and Ride-Sharing

Urban transportation plays an important role in providing access to employment, education, services and other daily activities. In many urban areas, commuters depend on a combination of private vehicles and public or informal transport services to reach their destinations. The availability, coverage and flexibility of these transport options can influence how easily people are able to travel between different locations. Cervero & Golub (2007) explain that informal transport systems can provide important mobility services, particularly where formal public transport does not adequately meet travel demand. However, informal transport can also experience challenges associated with coordination, service coverage and the organisation of transport services.

These issues are relevant to the transportation environment in Botswana, where commuters may rely on different forms of transport depending on their location and destination. Local reporting has, for example, documented concerns from combi operators in Francistown regarding the extension of routes to better serve areas such as the industrial area (Kebareng 2013). Although this example does not represent the entire transportation system in Botswana, it demonstrates how route configuration and access to particular destinations can affect local transport operations. The need to connect people with destinations therefore extends beyond simply having vehicles available; the routes and travel patterns of both transport providers and passengers are also important.

Ride-sharing provides an alternative approach to transportation in which individuals travelling in similar directions can share a vehicle. Unlike conventional public transport, where vehicles generally follow predetermined routes, ride-sharing can use the journeys of individual vehicle owners to create flexible transportation opportunities. Agatz et al. (2012) describe dynamic ride-sharing as a process in which drivers and passengers with

similar itineraries and schedules can be brought together, allowing available vehicle capacity to be used more effectively. This makes ride-sharing particularly relevant to situations where private vehicles may have unused seating capacity.

The potential benefits of ride-sharing extend beyond simply providing another method of transportation. Furuhata et al. (2013) identify reduced travel costs, improved utilisation of vehicle capacity and possible reductions in traffic and environmental impacts among the potential benefits associated with ridesharing. However, these benefits depend on the ability of a system to identify people whose journeys are sufficiently compatible. A driver travelling in one direction cannot necessarily accommodate a passenger travelling in another direction without introducing additional distance or time into the journey.

Consequently, an effective ride-sharing system must consider more than the availability of a vehicle. It must consider factors such as the driver's origin and destination, the passenger's origin and destination, departure times, available seats and the extent to which the two journeys can be combined. Stiglic et al. (2016) demonstrate that driver and passenger flexibility can have an important influence on the number of successful ridesharing matches. This indicates that the effectiveness of a ride-sharing system is strongly connected to the way in which potential journeys are identified and matched.

The proposed project therefore focuses on a peer-to-peer ride-sharing model for urban commuting in Botswana. The intended model allows a private vehicle owner who is already travelling towards a destination to offer available seats to passengers travelling along a compatible route. Instead of functioning primarily as an on-demand transport service in which a driver is expected to travel specifically to collect a passenger, the proposed system is intended to make use of journeys that are already taking place. The central problem addressed by the project is therefore the difficulty of efficiently connecting available vehicle capacity with passengers whose journeys are sufficiently compatible to be shared.

This creates an opportunity for a digital platform to assist with the coordination of these journeys. By allowing users to provide their locations, destinations, travel times and seating requirements, the application can use this information to identify potentially suitable rides. The following section examines existing ride-sharing and ride-hailing platforms to determine which approaches are already being used and which aspects can inform the design of the proposed system.

2.2 Existing Ride-Sharing and Ride-Hailing Systems

The development of digital transportation platforms has demonstrated that mobile applications can be used to coordinate passengers, drivers, locations and journeys. Existing systems provide useful examples of how users can request transportation, identify

available drivers, communicate journey information and manage trips. However, these systems do not all use the same transportation model. Understanding these differences is important because the proposed application is not intended to simply reproduce an existing ride-hailing platform.

Uber provides an example of a conventional on-demand ride-hailing model. A passenger enters a pickup location and destination, submits a ride request, and is matched with a nearby driver who accepts the request. The application can then provide information about the driver's arrival, the trip and navigation, while users can rate one another after the journey (Uber n.d.). These features demonstrate the usefulness of combining user accounts, geolocation, ride requests, trip information and status updates within one mobile platform. The proposed system can therefore take inspiration from Uber's digital interaction between passengers and drivers, particularly its use of location information and a structured ride-request process.

However, the transportation model is different from the one proposed in this project. In a conventional ride-hailing system, the driver accepts a passenger's request and travels to the passenger's pickup location. The proposed system instead focuses on a driver who already has a journey planned and has available seats. Therefore, the central question in the proposed system is not simply whether a driver is available near a passenger, but whether an existing driver's journey is sufficiently compatible with the passenger's requested journey.

Bolt provides another example of an app-based ride-hailing platform. Bolt describes its service as connecting passengers with drivers and allows users to enter journey information, receive ride options and request transportation. The platform also supports scheduled rides in advance and provides estimated prices based on factors such as journey distance and selected ride type (Bolt n.d.). From a system-design perspective, Bolt demonstrates how a transportation application can combine location-based requests, driver availability and journey information within a mobile interface. These concepts are relevant to the proposed application's user interface and ride-request workflow.

Nevertheless, Bolt remains primarily a ride-hailing service in which a passenger requests transportation from an available driver. The proposed application instead aims to coordinate shared journeys between private vehicle owners and passengers. This distinction is important because the matching mechanism must prioritise route compatibility and available seating rather than simply finding an available nearby vehicle.

inDrive demonstrates a somewhat different approach to the interaction between passengers and drivers. Users can specify a pickup location, destination and proposed price, after which drivers can respond to the request. Drivers can accept suitable requests,

decline them or make counter-offers, while passengers can consider different driver responses before selecting a driver (inDrive n.d.). This provides a useful example of allowing participants to have greater involvement in the process of arranging a journey rather than relying entirely on automatic assignment.

For the proposed system, the relevant concept is the interaction between a ride request and potential drivers. The proposed application can similarly use a request-and-response process in which a passenger requests a compatible ride and a driver can accept or decline the request. However, the proposed system does not intend to reproduce inDrive's price-negotiation model. Instead, the main consideration is whether the passenger's journey can be accommodated within the driver's existing route and available seating.

BlaBlaCar is particularly relevant because its model is closer to the peer-to-peer ride-sharing approach proposed in this project. BlaBlaCar connects drivers travelling towards a destination with passengers travelling in the same direction so that they can share the journey and associated costs. Drivers can publish journeys by specifying information such as departure and arrival locations, dates, times and available seats, while passengers can search for suitable journeys using criteria such as origin, destination and travel date (BlaBlaCar n.d.).

The BlaBlaCar model therefore provides a strong conceptual foundation for the proposed application. In particular, the concepts of available seats, shared journeys, compatible destinations and cost contribution are directly relevant to the project. BlaBlaCar has also developed approaches for shorter commuting journeys in which drivers and passengers can be matched according to similar itineraries and schedules, demonstrating the relevance of route and time compatibility to regular commuting (BlaBlaCar n.d.).

However, the proposed system is not intended to reproduce BlaBlaCar as a complete platform. The project focuses specifically on urban commuting in Botswana and is designed around a simplified peer-to-peer model suitable for the scope of the prototype. The system therefore takes inspiration from BlaBlaCar's core concept of connecting drivers with available seats to passengers travelling in a similar direction, while adapting the concept to the project's intended local commuting context.

Comparison of Existing Systems

Table 2.1: Comparison of existing ride-sharing and ride-hailing systems

Existing System	**Relevant features and approach**	**Relevance to the proposed system**
Uber	On-demand ride requests, pickup and destination selection, driver matching, trip information, navigation and post-trip ratings.	Informs the use of user accounts, geolocation, ride requests, trip status and interaction between drivers and passengers. The proposed system differs by focusing on drivers who are already travelling and have available seats.
Bolt	Mobile ride-hailing, location-based driver connection, journey information, ride options and scheduled trips.	Informs the request-and-response interaction between passengers and drivers. The proposed system instead prioritises route compatibility and available seats rather than price negotiation.
inDrive	Passengers provide journey details and proposed prices, while drivers can respond, accept, decline or make counteroffers.	Informs the request-and-response interaction between passengers and drivers. The proposed system instead prioritises route compatibility and available seats rather than price negotiation.
BlaBlaCar	Drivers publish journeys with available seats and passengers search for compatible journeys based on origin, destination, time and available seats. Costs are shared between participants.	Provides the closest conceptual model for the proposed system. The project adapts the available-seat and shared-journey approach for an urban commuting application focused on Botswana.

The comparison demonstrates that existing platforms provide several useful components for the proposed system, but they approach transportation in different ways. Uber and Bolt demonstrate the effectiveness of location-based on-demand ride requests, while inDrive demonstrates participant involvement in arranging rides. BlaBlaCar provides the closest example of the proposed peer-to-peer carpooling model because it connects drivers with available seats to passengers travelling in the same direction.

At the same time, the comparison identifies an important design opportunity. The proposed application is intended to combine selected concepts from these systems without becoming a conventional ride-hailing platform. Its main focus is on existing journeys, available vehicle capacity and route compatibility. A driver is therefore not primarily treated as a transport provider waiting for a passenger request; instead, the driver can

offer seats on a journey they already intend to make. This distinction gives the system a different operational focus from conventional ride-hailing.

The proposed system is consequently intended to provide a localised alternative centred on peer-to-peer commuting and cost sharing. Its value proposition is not based on assuming that users will automatically prefer it or that it will always be cheaper than other transportation options. Rather, the system is designed to provide an alternative way of coordinating journeys by allowing available vehicle capacity to be shared between people travelling along compatible routes. Whether users consider this approach useful or preferable would need to be evaluated through system testing and user feedback.

The comparison also shows that a successful implementation cannot rely only on identifying nearby users. A passenger may be geographically close to a driver while travelling towards a completely different destination. Conversely, users may begin from different locations while travelling along a substantial portion of the same route. Therefore, the proposed application requires a matching mechanism capable of considering the relationship between complete journeys. The literature on ride-matching provides the basis for determining which factors should be considered, as discussed in the following section.

2.3 Ride-Matching and Compatibility

Ride-matching is the process of identifying drivers and passengers whose journeys are sufficiently compatible to be combined into a shared trip. It is a central component of a dynamic ride-sharing system because the usefulness of such a system depends on its ability to identify suitable journeys rather than simply connecting an available driver with any passenger. Agatz et al. (2012) describe dynamic ride-sharing as a process of bringing together individuals with similar itineraries and schedules, while considering the available capacity of participating vehicles.

A basic matching process can consider the origin and destination of both the driver and passenger. However, comparing only the starting locations is insufficient for determining whether two journeys are suitable for sharing. Two users may be located close to each other but travel towards different destinations, while users starting from different locations may travel along a similar route for a substantial part of their journeys. Therefore, ride-matching needs to consider the compatibility of the complete journeys rather than relying only on geographical proximity.

Route compatibility is an important consideration because the driver's original journey should not be changed substantially simply to accommodate a passenger. Hong et al. (2017) demonstrate the importance of route information in ridesharing by developing a trajectory-based matching method that identifies common paths and sub-paths between vehicle journeys. Their research demonstrates that journeys sharing common route

segments can provide potential carpooling opportunities. This suggests that the degree to which two journeys follow similar routes can provide more useful information for matching than the physical distance between users alone.

For example, a driver travelling from one area of Gaborone to another may have an available seat, while a passenger located several kilometres away may be travelling towards the same general destination. The passenger could potentially be a suitable match if the driver's journey passes sufficiently close to the passenger's pickup location and continues towards the passenger's destination. However, if collecting the passenger requires the driver to travel substantially away from the original journey, the match becomes less practical even though the two destinations may be similar. This demonstrates why route compatibility should be considered when identifying potential rides.

Time compatibility is another important factor. Two journeys may have similar origins and destinations but still be unsuitable if their departure times are significantly different. A driver travelling at 07:00 cannot normally accommodate a passenger who needs to leave at 10:00 unless the driver is willing to change the planned departure time. Stiglic et al. (2016) investigated the effect of driver and rider flexibility on dynamic ride-sharing and found that flexibility in departure times and acceptable detours can influence the number of successful matches. These findings indicate that matching should consider when participants intend to travel as well as where they intend to travel.

Available vehicle capacity must also be considered. A driver may be travelling along a highly compatible route but have no remaining seats available. Similarly, a driver with several available seats may be able to accommodate multiple passengers, provided that their journeys remain compatible with the driver's route and the system's constraints. The number of available seats should therefore form part of the information used when determining whether a ride request can be accommodated.

The amount of additional travel required to accommodate a passenger is also relevant to the quality of a potential match. A passenger may have a compatible destination but require a pickup location that causes the driver to make a significant detour. Although the journey may technically be possible, the additional distance or travel time may make the match undesirable for the driver. Stiglic et al. (2016) identify detour flexibility as an important factor in dynamic ride-sharing performance. Therefore, an acceptable match should consider not only whether a passenger can be collected, but also the amount of additional travel required.

Ride-matching can also involve participant preferences. Wang et al. (2018) examine stable matching for dynamic ride-sharing systems and consider the relationship between individual preferences and system-level outcomes. Their work demonstrates that identifying a technically possible pairing does not necessarily mean that the pairing is desirable to the participants. For example, a driver and passenger may have compatible destinations and departure times, but the passenger's requested pickup point may require

a large detour. The driver may therefore reject the match even though it satisfies the basic geographical and temporal requirements. Similarly, a passenger may have several compatible drivers available but prefer the option requiring less additional travel.

These considerations indicate that ride-matching should involve a combination of constraints rather than a single measure. A potential match for the proposed system can consider factors such as the driver's and passenger's origins and destinations, departure-time compatibility, route compatibility, available seating and the additional travel required to accommodate the passenger. These factors can first be used to eliminate clearly unsuitable journeys and can then be used to identify or rank the remaining potential matches.

For the proposed application, the matching process is intended to support the peer-to-peer model established in the previous section. A driver will be able to offer available seats on a planned journey, while a passenger will provide information about their requested journey. The system can then compare the relevant journey information to identify potentially compatible rides. This means that the matching process is not primarily concerned with finding the geographically nearest driver. Instead, it is concerned with determining whether an existing driver's journey can reasonably accommodate the passenger's journey.

The literature also suggests that matching complexity can vary depending on the approach used. Basic matching can use predefined constraints such as acceptable departure-time differences, available seats and maximum detour limits. More advanced approaches can use route trajectories, optimisation techniques or preference-based matching. Wang et al. (2018) demonstrate that stable matching can be used to account for participant preferences and system-level outcomes, while Hong et al. (2017) demonstrate the use of trajectory information to identify common travel paths. However, implementing highly complex optimisation techniques may be beyond the requirements of the proposed prototype.

The proposed system will therefore focus on a practical and explainable matching approach appropriate to the project's scope. The matching mechanism can use measurable criteria to identify journeys that satisfy the required spatial, temporal and capacity constraints. The exact thresholds, calculations and matching procedure can be defined during the system design and implementation stages and subsequently evaluated through testing.

Overall, the literature indicates that effective ridesharing requires more than identifying users who are geographically close. A suitable match depends on the relationship between the driver's and passenger's journeys, including their origins, destinations, travel times, available seating, route compatibility and acceptable additional travel. These findings provide the conceptual basis for the proposed ride-matching mechanism. The next section examines geolocation and route calculation as the technical means through

which the application can obtain the location and route information required by this matching process.

2.4 Geolocation and Route Matching

Geolocation refers to the representation of the position of an object within a geographical context (Gustafsson 2014). Geolocation provides the geographical information required to show the positions of drivers and passengers. Mobile devices can determine their geographical position using Global Navigation Satellite System (GNSS) technologies, including the Global Positioning System (GPS). Smartphone-based positioning is widely used across various applications, especially navigation applications; however, the accuracy of smartphone positioning can be affected by factors such as signal conditions like strength, the geography of the environment and device characteristics (such as the quality of the smartphone's positioning receiver and antenna) (Zangenehnejad & Gao 2021). The location information obtained from the mobile device should therefore be considered as an estimate of the user's physical position rather than a perfectly exact measurement.

Geographical position can be represented using latitude and longitude coordinates. These coordinates provide a numerical representation of a location and allow geographical information to be stored and processed by software. In the application, the current location of a user can be obtained from the mobile device and be represented as a pair of geographic coordinates. The intended destination can also be represented as a geographical location. This provides a consistent representation of the origins and destinations required for route calculations.

In the case of the ridesharing app, coordinates alone cannot sufficiently describe the journey that a vehicle follows. That is, two locations may be geographically close while being separated by roads, unnavigable areas or other obstacles. A routing service is therefore required to determine a practical route between specified locations by accepting geographical locations (origin and destination) and calculating a route through the available road network.

The application could use the Google Maps Routes API to perform the route calculation. The Routes API provides the `computeRoutes()` method, which accepts an origin and destination and can return information about the calculated route. The returned information can include the route distance, travel duration and an encoded polyline representing the route geometry (Google 2026*b*). The use of such a routing system would allow the application to obtain journey information based on road networks rather than relying on the geographical distance between two locations/coordinates.

The origin and destination given to a routing service can be represented using latitude and longitude coordinates. The route calculation then produces a sequence of connected road segments between the specified locations, and the Google Maps Routes API represents the route information in the form of distance (in metres), duration and a polyline (Google 2026*b*). This information provides a structured representation of the journey that can be processed by the application.

Route distance provides a measurable representation of the length of a road journey, while estimated duration provides an approximation of the expected time required to travel along the calculated route. These values are different from straight-line geographical distance because they are based on the road network. For example, two locations may be relatively close when measured directly between their coordinates but require a longer journey because the available roads do not provide a direct connection. This makes road-based distance more appropriate for representing vehicle journeys within the application.

The route polyline provides additional geographical information by representing the geometry of a calculated route. This allows the route to be displayed on a map and provides a representation of the journey between the origin and destination. The route geometry can also provide geographical information that may then be processed by the ride-matching mechanism. The Google Maps Routes API identifies encoded polylines as one of the route fields that can be returned by a route calculation (Google 2026*b*).

The application can process geographical information at several levels. The mobile device provides the user's current location, represented using geographic coordinates. The user provides or selects a destination, which is also represented as coordinates. The routing service then uses the origin and destination to calculate a road route and provide measurable information such as distance and duration. The resulting route information can then be passed over to the ride-matching system so that the compatibility of the journeys can be determined.

The use of smartphone-based positioning introduces a level of uncertainty to the geographical information used by the application. Research on smartphone GNSS positioning has identified challenges associated with signal conditions, device characteristics, environment and noisy observations. These limitations are important to keep in mind because inaccurate location information could affect the position reported for a driver or passenger. For instance, a reported position may not correspond exactly to the user's physical position. The program should therefore treat obtained coordinates as location estimates and apply reasonable distance thresholds rather than assuming perfect positional accuracy.

Using a routing service introduces an external technical dependency. The application must communicate with the mapping service to obtain calculated routes and related information. The route calculation depends on network connectivity, API availability

and configuration of the mapping service. The Google Maps Routes API also requires the response information to be specified through field masks, allowing applications to request route information such as duration, distance and encoded polyline data (Google 2026*b*). These dependencies are relevant to the reliability and performance of the application.

For systems that involve multiple possible matches, routing services can provide mechanisms for calculating information between multiple origins and destinations. The Google Maps Routes API includes the `computeRouteMatrix()` method, which can calculate route information for combinations of origins and destinations and return values such as distance and duration (Google 2026*a*). While such functionality is highly useful when evaluating multiple driver–passenger combinations, the proposed prototype initially limits calculations strictly to candidate journeys required by the matching process to minimise unnecessary API overhead.

2.5 Factors Affecting Ride Compatibility

The effectiveness of a dynamic ridesharing system depends on its ability to identify journeys that are sufficiently compatible for a driver and passenger to share a trip. Compatibility cannot be determined using a single factor because a suitable match depends on several factors, such as the geographical relationship between journeys, the time at which the journeys take place, the amount of additional travel required and the available vehicle capacity. Dynamic ridesharing therefore considers multiple characteristics when determining whether travellers can be matched.

An important factor is the distance between the passenger's pickup location and the driver's planned route. A passenger who is located close to the driver's route can generally be accommodated with less additional travel than a passenger who is located far from the route. Pickup distance can therefore be used to determine whether collecting a passenger would require an excessive deviation from the driver's planned journey. A smaller pickup distance would be considered a more practical match, while a larger distance may reduce the suitability of the journey route.

Route overlap is another important factor in determining compatibility. Two journeys may have different origins but still share a substantial portion of the same route. Journeys like these provide ridesharing potential because the driver can transport the passenger while continuing along a similar route. The relevance of common route segments is identified by using road network topology and vehicle trajectories to find common sub-paths between journeys (Hong et al. 2017). This indicates that the extent of two journeys sharing a route can provide a stronger indication of compatibility than just comparing the locations of the users.

Destination compatibility is another factor that must be considered. A passenger's destination should be sufficiently related to the driver's intended destination or route for the shared journey to be practical. For example, a passenger travelling to a location along the driver's route may be more suitable than a passenger whose destination requires the driver to leave their planned route. Destination compatibility can therefore be considered along with route overlap to determine whether the passenger can be accommodated without substantially changing the driver's journey.

Departure time is also a significant factor in dynamic ridesharing. Even when two journeys have highly compatible routes, they cannot form a practical match if the journeys occur at marginally different times. Dynamic ride-sharing is a system in which travellers with similar itineraries and time schedules can be brought together, highlighting the importance of coordinating both spatial and temporal characteristics (Agatz et al. 2012). Furthermore, flexibility in departure time can influence matching performance, as demonstrated by Stiglic et al. (2016). This suggests that the system should consider the requested departure time of the passenger in relation to the driver's planned departure time when evaluating potential matches.

Driver detour is related to both route and pickup compatibility. A potential passenger may have a suitable destination but still require the driver to travel considerably farther than originally planned to collect or drop off the passenger. Such additional travel reduces the efficiency of the shared journey and may make the match unacceptable for the driver (Stiglic et al. 2016). The amount of additional distance or time introduced by the passenger can therefore be used as an indicator of the practicality of a potential match.

Vehicle capacity provides another necessary constraint. A driver may have a route that is highly compatible with several passengers, but the number of passengers that can be accepted is limited by the available seats in the vehicle. In a ridesharing application, available seating must be considered when identifying and accepting matches. A ride should not be considered available to additional passengers once its available seating capacity has been reached.

Participant flexibility can also affect the number of potential matches available. Allowing greater flexibility in departure time and driver detour can increase the likelihood of establishing successful matches (Stiglic et al. 2016). However, excessive flexibility may reduce the attractiveness of a ride because users may have to accept substantial changes to their original journeys. A practical ridesharing system must therefore balance the need to increase matching opportunities with the need to maintain reasonable travel conditions for both drivers and passengers.

The factors discussed above are not independent of one another. A journey with high route overlap may still be unsuitable if the passenger requires a large pickup detour or departs at a significantly different time. Similarly, a journey with a convenient pickup

location may not be suitable if the driver's destination differs significantly from the passenger's. Effective matching therefore requires multiple factors to be considered rather than relying on a single measure of similarity, as they provide measurable characteristics that can be used to distinguish stronger matches from weaker ones rather than treating all rides as equally suitable.

Chapter 3

System Analysis and Design

This chapter explains the design of the ride-sharing application through its requirement analysis, high-level architecture, matching process, database design and class diagrams. Where a decision responds to a finding from the background literature (Chapter 2), the finding, the decision and the source are stated together.

3.1 Requirement Analysis

The system requirements were analysed using Use Case modelling to identify the interactions between the user actors (Driver and Passenger) and the system. Only the major and complex use cases are detailed below, as they directly inform the identification of the system's core classes and components.

3.1.1 Major Use Case Scenarios

Use Case 1: Requesting a Ride and Matching (Complex)

- **Actor:** Passenger
- **Preconditions:** The passenger is authenticated and has submitted a ride request (RQ04).
- **Main Flow:**
  1. The system fetches all active driver rides from the database.
  2. For each candidate ride, the system calculates the route overlap and detour distance using the Google Maps Routes API.
  3. The system applies the matching algorithm to filter out rides that exceed the maximum detour or time window thresholds.

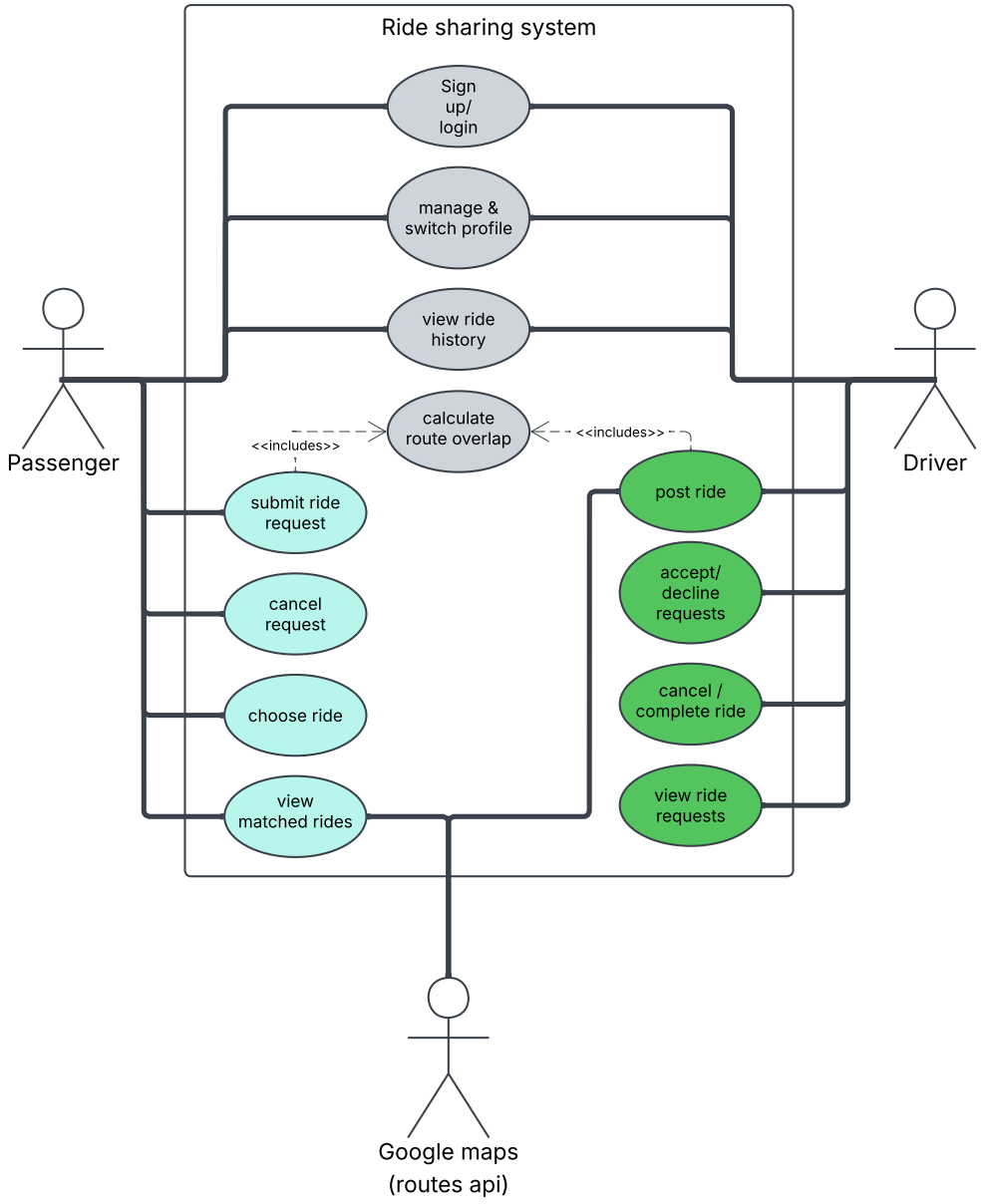


FIGURE 3.1: Use Case Diagram for the ride-sharing system

4. Compatible rides are scored and ranked based on proximity, time compatibility, and route overlap (RQ05).
5. The system returns the ranked list of matches to the passenger.

- **Postconditions:** A list of `MatchResult` records is generated and displayed to the passenger.

Use Case 2: Posting a Ride (Major)

- **Actor:** Driver

- **Preconditions:** The driver is authenticated.
- **Main Flow:**
  1. The driver inputs the origin, destination, departure time, and available seats.
  2. The system calls the Google Maps Routes API to calculate the route geometry, distance, and estimated duration.
  3. The system stores the ride offer and the calculated route data in the database (RQ03).
- **Postconditions:** A new `Ride` record is created and made visible to passengers.

3.1.2 Identification of Classes from Use Cases

The analysis of the above use cases directly informed the system's class structure. For example, the "Requesting a Ride" use case involves comparing a passenger's journey against multiple driver journeys, which identified the need for a dedicated `MatchingService` class to handle the computation, and a `MatchResult` class to store the output. Similarly, the "Posting a Ride" use case requires external API calls, identifying the need for a `RouteService` class to encapsulate the Google Maps API communication.

3.2 Overview of Design

3.2.1 System Architecture

The high-level architecture is shown in Figure 3.2. The system has three parts: the Expo client application on the user's mobile device; the Supabase backend, which provides authentication, data storage and server-side logic; and the Google Maps Routes API (Application Programming Interface), an external service that supplies road-network route information.

The client and the backend communicate over REST (Representational State Transfer) for request–response calls, such as posting a ride, and over a realtime channel that pushes changes, such as the status of a request, to the client (Supabase 2026). Unlike the on-demand ride-hailing platforms reviewed in Section 2.2, where a driver is assigned after a passenger requests a ride, the drivers in this system have already planned their journeys. Each journey is therefore stored in advance as a ride offer, and the backend compares complete journeys when a passenger submits a request (Section 2.3). This requires the server-side matching and routing components described below.

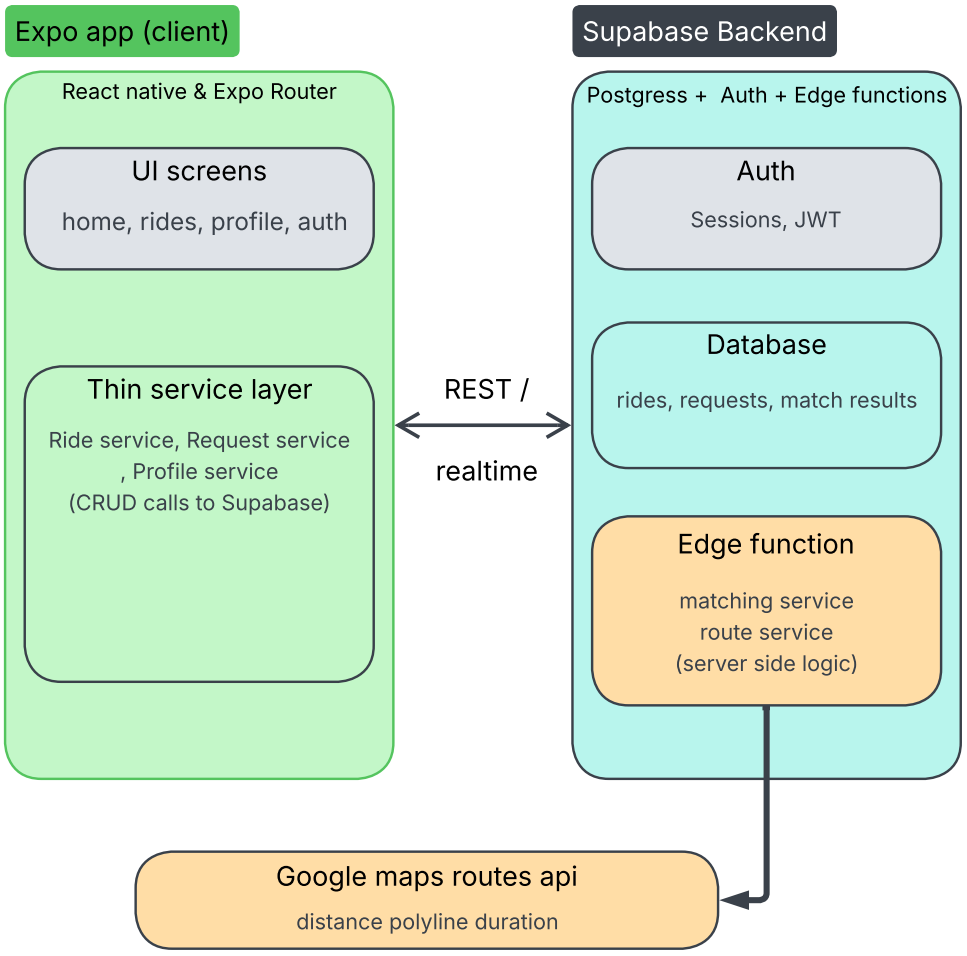


FIGURE 3.2: High-level architecture of the ride-sharing system

- **Finding:** On-demand ride-hailing platforms suffer from "deadheading," where drivers travel empty to pick up a passenger, increasing costs and congestion (Furuhata et al. 2013).
- **Decision:** Unlike on-demand systems, drivers in this application have pre-planned their journeys. Each journey is stored in advance as a ride offer, and the backend compares complete journeys only when a passenger submits a request.
- **Reference:** This design eliminates deadheading, aligning with the peer-to-peer cost-sharing model defined by Furuhata et al. (2013).

3.2.2 Architectural Components

Expo client. The client uses React Native and Expo Router so that one code base serves both Android and iOS (Expo 2026, Meta Platforms, Inc. 2026), which addresses RQ14. Its user interface (UI) screens (home, rides, profile and authentication) let users

register and sign in, switch role, post, search and request rides, respond to requests, view status and history, and select locations on the interactive map (RQ01–RQ04, RQ06–RQ09, RQ11). The screens contain presentation logic only. Between the screens and the backend sits a thin service layer (Ride, Request and Profile services), which forwards each action to Supabase as a CRUD (Create, Read, Update, Delete) call. This separation supports the modularity requirement (NFR08), because a change in how data is stored does not require changes to the screens.

Supabase backend. The backend is built on a PostgreSQL database and provides authentication, storage and server-side functions without a separate server application (Supabase 2026). Auth manages registration, sign-in and sessions; a JWT (JSON Web Token) issued at sign-in identifies the user in later requests, so protected functionality and data are restricted to the authenticated user and their role (RQ01, RQ13, NFR06). The database stores profiles, rides, requests and match results and supports ride histories (RQ08, RQ12).

Edge function. The Edge function holds the matching service and the route service. Matching compares a passenger's journey with several candidate rides using route information from an external API (Section 2.3; Google 2026*b*), so this logic was placed on the server and not in the client. This ensures that the same matching criteria apply on every device (RQ15, NFR08) and keeps the credentials for the routing service on the server, off the mobile device (NFR06).

Google Maps Routes API. The Routes API returns the route distance, the estimated duration and an encoded polyline of the route geometry (Google 2026*b*). The distance between two coordinates does not represent a vehicle journey, whereas road-based values do (Section 2.4), and route overlap indicates compatibility better than user locations alone (Hong et al. 2017). Route information is therefore obtained from the API and not calculated from coordinates. Because each call adds overhead and the system depends on the service being available (NFR07), route calculations are limited to the journeys the matching process needs (Section 2.4; Google 2026*a*).

- **Finding:** Calculating road-based route overlap indicates compatibility significantly better than straight-line user locations (Hong et al. 2017).
- **Decision:** The matching logic was placed on the server-side Edge function rather than the client, and it utilises the Google Maps Routes API to obtain accurate route geometry.

- **Reference:** This ensures consistent matching criteria across all devices (RQ15) and keeps external API credentials secure on the server (RQ13), as recommended by Google (2026*b*).

3.2.3 Matching Process

The matching process is shown in Figure 3.3. When a passenger submits a request (RQ04), the active driver rides are fetched as candidates. The route of each candidate is compared with the passenger's journey (Hong et al. 2017) and the ride is added to the match list if the overlap meets the threshold, or discarded if it does not. The comparison repeats until every candidate has been evaluated, and the matches are then ranked and returned (RQ05, RQ15). Unsuitable rides are therefore eliminated first and the remainder ranked, as described in Section 2.3.

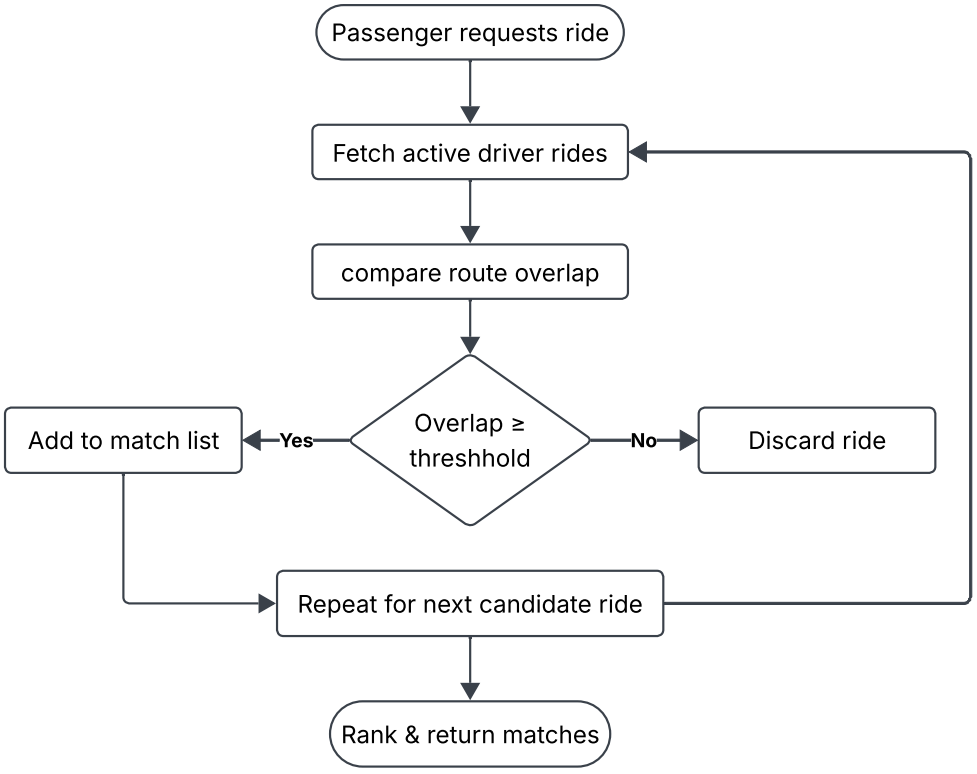


FIGURE 3.3: Flow chart of the ride-matching process

In the class design (Section 3.4.2), the overlap test is realised by `MatchingService`: a ride is accepted only when it has enough free seats and its pickup distance, detour and difference in departure time are within set limits. Accepted rides are scored and returned as `MatchResult` records.

3.3 Entity Relationship Diagram (ERD)

The database layout is illustrated in Figure 3.4. All tables required for the system are shown in the diagram, but only the major tables addressing core requirements are discussed below.

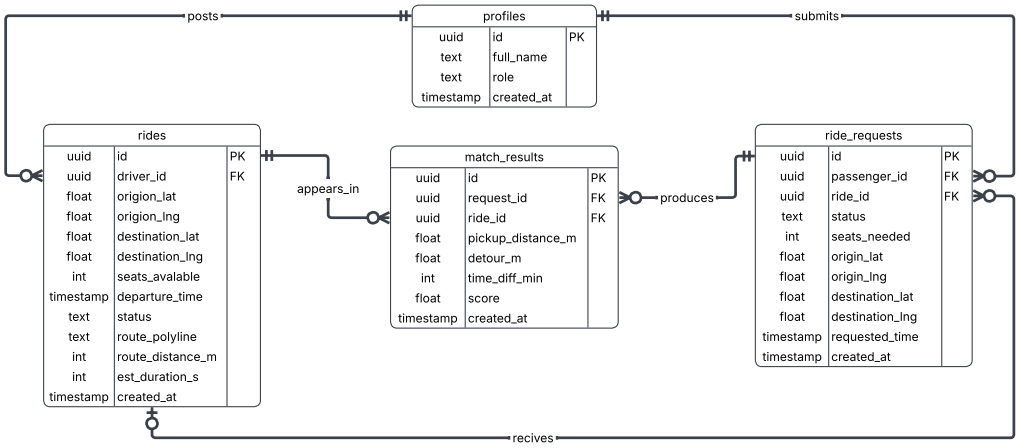


FIGURE 3.4: Entity Relationship Diagram (ERD) of the database

- **Profile Table:** Represents a registered user (RQ02). It stores personal details and the user's current role (Driver or Passenger).
- **Ride Table:** Represents a ride offer posted by a driver (RQ03). It stores the origin, destination, departure time, and `seatsAvailable`. Crucially, it also stores the `routePolyline`, `routeDistanceM`, and `estDurationS` obtained from the Routes API when the ride is posted. Storing this data prevents redundant API calls during the matching process.
- **RideRequest Table:** Represents a passenger's request to travel (RQ04). It stores the requested origin, destination, time, and the current status of the request (e.g., Pending, Accepted, Declined).
- **MatchResult Table:** Stores the calculated compatibility scores between a specific `RideRequest` and candidate `Ride` records (RQ05).

The relationships are strictly defined: one `Profile` can post zero or many `Ride` records (1 to 0..*), and one `Profile` can submit zero or many `RideRequest` records (1 to 0..*). A `Ride` can receive zero or many `RideRequest` records, but the number of accepted requests is constrained by the `seatsAvailable` attribute.

3.4 Class Diagrams

The classes are divided into entity classes, which represent stored data, and service classes, which hold the behaviour that creates, retrieves and processes it. The entity classes correspond to the database records in Figure 3.4, and the service classes correspond to the thin service layer and the Edge function. The diagrams use UML (Unified Modelling Language) notation, in which `+` and `-` mark public and private members and multiplicities show how many objects of one class relate to one object of another. Parameter types are shown in the figures and omitted from the tables.

3.4.1 Entity Classes

Figure 3.5 shows the four entity classes. Every entity has an `id` of type UUID (Universally Unique Identifier), locations are stored as latitude and longitude pairs (Section 2.4), and a suffix gives the unit of a measured value (`M` for metres, `S` for seconds, `Min` for minutes).

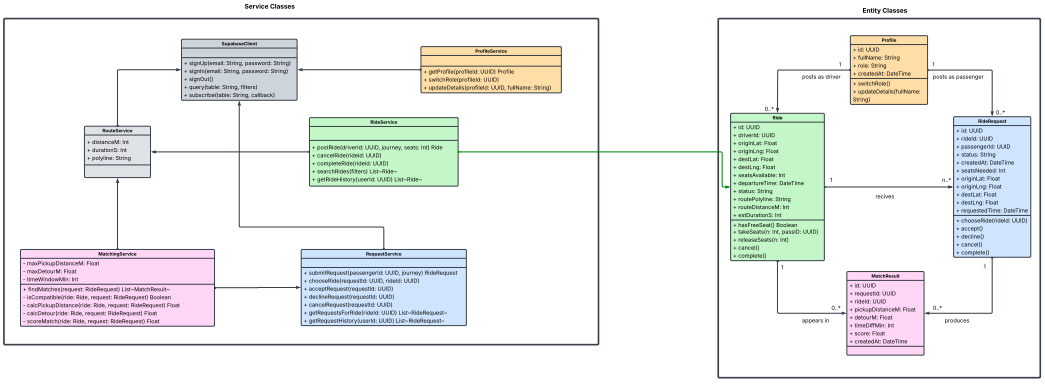


FIGURE 3.5: Entity classes of the system

Profile. `Profile` represents a registered user (RQ02). One profile serves both drivers and passengers, with `role` recording the current role, so a user does not need to register twice. Credentials are not stored here because they are handled by Auth (RQ01).

Ride. `Ride` is a ride offer posted by a driver for a planned journey (RQ03) and is the central entity for matching, search and history. It stores the origin, destination, departure time and `seatsAvailable`, together with the route (`routePolyline`, `routeDistanceM`, `estDurationS`) obtained from the Routes API when the ride is posted. Storing the route avoids requesting it again for every passenger request, which reduces external calls during matching (NFR01).

RideRequest. `RideRequest` is a passenger's request to travel (RQ04). It stores the origin, destination and `requestedTime` used in matching, the `seatsNeeded` and the `status`. The `rideId` records the ride the passenger chooses and is set through `chooseRide()`. The operations `accept()`, `decline()`, `cancel()` and `complete()` represent the life cycle of a request (RQ06, RQ07).

MatchResult. `MatchResult` records the outcome of comparing one request with one candidate ride (RQ05, RQ15). Its measurements, `pickupDistanceM`, `detourM` and `timeDiffMin`, correspond to compatibility factors identified in the literature (Stiglic et al. 2016; Section 2.5) and are combined into a score for ranking. It has no operations because it is a record; the behaviour that produces it belongs to `MatchingService`.

The operations of the entity classes are summarised in Table 3.1. The seat operations ensure that a ride cannot be allocated more passengers than the seats it offers (NFR03).

Table 3.1: Operations of the entity classes

Class	Operation	Description
Profile	`switchRole()`	Changes the active role between driver and passenger.
Profile	`updateDetails(fullName)`	Updates the user's personal details.
Ride	`hasFreeSeat()`	Returns whether at least one seat remains; full rides are not offered to further passengers.
Ride	`takeSeats(n, passID)`	Allocates n seats to a passenger and reduces `seatsAvailable`.
Ride	`releaseSeats(n)`	Returns n seats, for example when a request is cancelled.
Ride	`cancel()`, `complete()`	Set the status of the ride to cancelled or completed.
RideRequest	`chooseRide(rideId)`	Links the request to the ride selected by the passenger.
RideRequest	`accept()`, `decline()`	Record the driver's decision on the request.
RideRequest	`cancel()`, `complete()`	Set the status of the request to cancelled or completed.

The multiplicities in Figure 3.5 are as follows. A profile posts zero or more rides as a driver and submits zero or more requests as a passenger (1 to 0..* in both cases), and each ride and request belongs to exactly one profile. A ride may receive zero or more requests (1 to 0..*), although the number that can be accepted is limited by `seatsAvailable`. A request produces zero or more match results, one for each compatible ride, and a ride may appear in zero or more match results (1 to 0..* in both cases); each match result refers to exactly one request and one ride. These associations are implemented

through the identifier attributes `driverId`, `passengerId`, `rideId` and `requestId`, which act as references (foreign keys) between database records. To ensure all requirements are accounted for, these multiplicities are explicitly outlined to demonstrate that the system can handle multiple concurrent requests and matches without data loss, strictly adhering to the 1 to 0..* relationships defined above.

3.4.2 Service Classes

Figure 3.6 shows the service classes. An arrow means that the class at its tail uses the class at its head, and colours link each service to its entity (orange for `Profile`, green for `Ride`, blue for `RideRequest`, pink for `MatchResult`; grey for infrastructure). Matching is a process and not stored data, so it is modelled as a service class, `MatchingService`, with `MatchResult` storing its output. Ride-matching combines several constraints and cannot be reduced to a single measure (Agatz et al. 2012, Stiglic et al. 2016), and this class keeps them in one place.

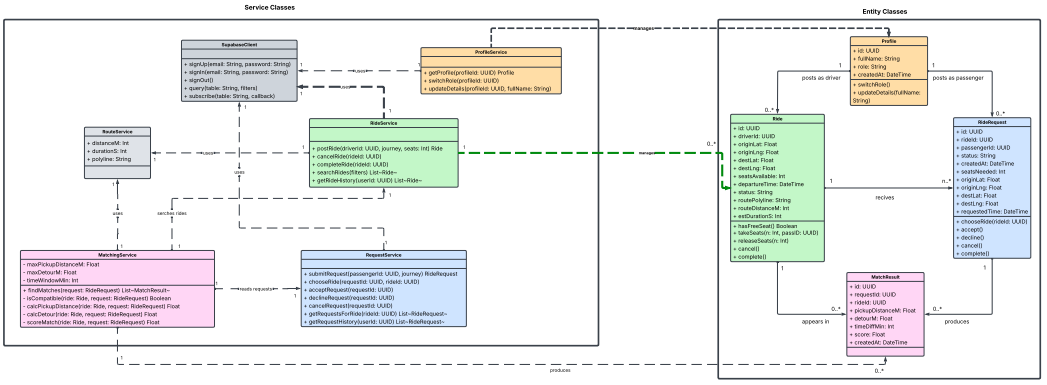


FIGURE 3.6: Service classes of the system

SupabaseClient and ProfileService. `SupabaseClient` is the single access point to the backend. Its operations `signUp()`, `signIn()` and `signOut()` handle authentication (RQ01, RQ13), `query()` reads records (RQ12), and `subscribe()` registers a callback for changes in a table so that status updates arrive in realtime (RQ07, RQ10). `ProfileService`, `RideService` and `RequestService` depend on it and not on the backend directly, so a change in backend access affects one class (NFR08). `RouteService` does not use `SupabaseClient` because it only communicates with the Google Maps Routes API. `ProfileService` retrieves and updates the signed-in user's profile through `SupabaseClient` (RQ02); its operations mirror those of `Profile` but locate the profile by `id` and save the change.

RideService and RequestService. These classes provide the operations used to work with ride offers and requests, listed in Table 3.2. `RideService` depends on

`RouteService` to obtain a route when a ride is posted; because `RouteService` runs on the server, this dependency crosses from the client to the backend over the REST channel. `MatchingService` depends on `RequestService` because matching is performed on stored requests.

Table 3.2: Operations of `RideService` and `RequestService`

Class	Operation	Description
RideService	`postRide(driverId, journey, seats)`	Returns the new `Ride`. Creates a ride offer from the driver's journey and seats and stores its route (RQ03).
RideService	`cancelRide(rideId)`, `completeRide(rideId)`	Cancel or complete a ride (RQ07).
RideService	`searchRides(filters)`	Returns available rides matching criteria such as destination, departure time and seats (RQ11).
RideService	`getRideHistory(userId)`	Returns the user's previous rides (RQ08).
RequestService	`submitRequest(passengerId, journey)`	Returns the new `RideRequest`; matching is then performed for it (RQ04, RQ05).
RequestService	`chooseRide(requestId, rideId)`	Links the request to the ride the passenger selected.
RequestService	`acceptRequest(requestId)`	Records the driver's acceptance; the seats are then allocated with `takeSeats()` (RQ06).
RequestService	`declineRequest(requestId)`	Records the driver's refusal (RQ06).
RequestService	`cancelRequest(requestId)`	Cancels the request; allocated seats are returned with `releaseSeats()`.
RequestService	`getRequestsForRide(rideId)`	Returns the requests for a ride so the driver can respond (RQ06).
RequestService	`getRequestHistory(userId)`	Returns the user's previous requests (RQ08).

MatchingService. `MatchingService` implements the process in Figure 3.3 (RQ05, RQ15). Its private attributes `maxPickupDistanceM`, `maxDetourM` and `timeWindowMin` are limits derived from the compatibility factors in Section 2.5. Flexibility in departure time and detour influences the number of matches, but too much flexibility reduces a ride's attractiveness (Stiglic et al. 2016), so the limits are adjustable attributes that can be tuned during testing (Section 2.3). Only `findMatches()` is public; the other operations are private internal steps (Table 3.3).

- **Finding:** Flexibility in departure time and detour influences match rates, but too much flexibility reduces a ride's attractiveness (Stiglic et al. 2016).
- **Decision:** `MatchingService` includes private attributes for `maxPickupDistance`, `maxDetour`, and `timeWindow` which act as strict filtering thresholds before scoring.
- **Reference:** This ensures the algorithm balances match quantity with driver convenience, as advised by Stiglic et al. (2016).

Table 3.3: Operations of `MatchingService`

Operation	Description
`+ findMatches(request)`	Returns a list of `MatchResult`. Evaluates each candidate ride, discards incompatible rides and returns the rest ranked by score.
`- isCompatible(ride, request)`	Returns true when enough seats are free for `seatsNeeded` and the pickup distance, detour and time difference are within the limits.
`- calcPickupDistance(ride, request)`	Returns the distance between the pick-up location and the route of the ride.
`- calcDetour(ride, request)`	Returns the additional distance the driver would travel to accommodate the passenger.
`- scoreMatch(ride, request)`	Combines the measurements into one score used for ranking.

RouteService. `RouteService` encapsulates communication with the Google Maps Routes API and holds the route values returned: `distanceM`, `durationS` and `polyline` (Google 2026*b*). `RideService` uses it when a ride is posted and `MatchingService` when routes are compared. Isolating the external service in one class means that only this class changes if the routing provider changes (NFR08).

3.4.3 Requirements Traceability

Table 3.4 shows how each requirement in the requirements catalogue (Appendix A) is realised in the design.

Table 3.4: Traceability of requirements to components and classes

ID	Requirement	Realised by
RQ01	Registration and authentication	Auth; `SupabaseClient` (`signUp`, `signIn`, `signOut`)
RQ02	User roles and profiles	`Profile`; `ProfileService`

ID	Requirement	Realised by
RQ03	Driver ride posting	`Ride`; `RideService.postRide`
RQ04	Passenger ride request	`RideRequest`; `RequestService.submitRequest`
RQ05	Ride matching	`MatchingService`; `MatchResult`; `RouteService`
RQ06	Ride request management	`RideRequest` (`accept`, `decline`); `RequestService`
RQ07	Ride status management	`status` attributes; cancel and complete operations; `SupabaseClient.subscribe`
RQ08	Ride history	`RideService.getRideHistory`; `RequestService.getRequestHistory`
RQ09	Interactive map	UI screens; coordinate attributes; `RouteService.polyline`
RQ10	Notifications	`SupabaseClient.subscribe` (realtime channel)
RQ11	Ride search and filtering	`RideService.searchRides`
RQ12	Data storage and retrieval	Database; `SupabaseClient.query`; all entity classes
RQ13	Access control	Auth (sessions, JWT); `Profile.role`
RQ14	Mobile application support	Expo client application (React Native)
RQ15	Ride matching results	`MatchingService.findMatches`; `MatchResult`
RQ16	Core task completion	UI screens (usability evaluated with the System Usability Scale)

Bibliography

Agatz, N., Erera, A., Savelsbergh, M. & Wang, X. (2012), 'Optimization for dynamic ride-sharing: A review', *European Journal of Operational Research* **223**(2), 295–303.

BlaBlaCar (n.d.), 'Todo: page title'. TODO: URL and access date.

Bolt (n.d.), 'Todo: page title'. TODO: URL and access date.

Cervero, R. & Golub, A. (2007), 'Informal transport: A global perspective', *Transport Policy* **14**(6), 445–457.

Expo (2026), 'Expo documentation', Available at: https://docs.expo.dev/. Accessed: 23 August 2026.

Furuhata, M., Dessouky, M., Ordóñez, F., Brunet, M.-E., Wang, X. & Koenig, S. (2013), 'Ridesharing: The state-of-the-art and future directions', *Transportation Research Part B: Methodological* **57**, 28–46.

Google (2026*a*), 'Routes API: Compute route matrix', Google for Developers. Available at: https://developers.google.com/maps/documentation/routes/reference/rest/v2/TopLevel/computeRouteMatrix. Accessed: 13 September 2026.

Google (2026*b*), 'Routes API: Compute routes', Google for Developers. Available at: https://developers.google.com/maps/documentation/routes/reference/rest/v2/TopLevel/computeRoutes. Accessed: 13 September 2026.

Gustafsson (2014), 'Todo: title of source on geolocation'. TODO: full reference.

Hong, Z., Chen, Y., Mahmassani, H. & Xu, S. (2017), 'Commuter ride-sharing using topology-based vehicle trajectory clustering: Methodology, application and impact evaluation', *Transportation Research Part C: Emerging Technologies* **85**, 573–590.

inDrive (n.d.), 'Todo: page title'. TODO: URL and access date.

Kebareng (2013), 'Todo: article title (report on francistown combi route extension)'. TODO: newspaper, URL and access date.

Meta Platforms, Inc. (2026), 'React native documentation', Available at: https://reactnative.dev/docs/getting-started. Accessed: 22 August 2026.

Stiglic, M., Agatz, N., Savelsbergh, M. & Gradisar, M. (2016), 'Making dynamic ride-sharing work: The impact of driver and rider flexibility', *Transportation Research Part E: Logistics and Transportation Review* **91**, 190–207.

Supabase (2026), 'Supabase documentation', Available at: https://supabase.com/docs. Accessed: 23 August 2026.

Uber (n.d.), 'Todo: page title'. TODO: URL and access date.

Wang, X., Agatz, N. & Erera, A. (2018), 'Stable matching for dynamic ride-sharing systems', *Transportation Science* **52**(4), 850–867.

Zangenehnejad, F. & Gao, Y. (2021), 'Gnss smartphones positioning: advances, challenges, opportunities, and future perspectives', *Satellite Navigation* **2**, 24.

Appendix A

Requirements Catalogue

Each requirement is listed with the non-functional requirements (NFR) that apply to it.

A.1 Functional Requirements

Requirement ID	RQ01
Requirement Description	**User Registration and Authentication** — Users shall be able to create and manage an account. The system shall allow users to register the required account information and authenticate their identities before accessing protected functionality. Authentication credentials shall be securely handled by the authentication service.
Non-functional Requirements	NFR01 Security — authentication credentials secured through appropriate access controls.

Requirement ID	RQ02
Requirement Description	**User Roles and Profiles** — Users shall be able to maintain a profile and interact with the system as a driver or passenger. The system shall allow an authenticated user to operate as a driver or passenger and shall store the relevant profile information associated with the user's account and role.
Non-functional Requirements	NFR01 Security — role-based access protection; NFR02 Reliability — profile and role data stored without inconsistency.

Requirement ID	RQ03
Requirement Description	**Driver Ride Posting** — Drivers shall be able to offer available seats on a planned journey. The system shall allow drivers to create and publish a ride offer by entering their origin, destination, departure time and number of available seats. The ride offer shall be stored and made available for matching and searching.
Non-functional Requirements	NFR02 Reliability — consistent seat availability data; NFR03 Performance — ride offers processed within a reasonable time.

Requirement ID	RQ04
Requirement Description	**Passenger Ride Request** — Passengers shall be able to request a ride for a planned journey. The system shall allow passengers to submit a ride request containing their origin, destination and relevant travel information. The request shall be stored and made available for matching suitable ride offers.
Non-functional Requirements	NFR02 Reliability — accurate request data handling; NFR03 Performance — timely request submission and storage.

Requirement ID	RQ05
Requirement Description	**Ride Matching** — Users shall be able to identify rides that are compatible with their journeys. The system shall compare relevant driver and passenger journey information, including origin, destination, departure time, available seating and route compatibility, and shall identify potential compatible rides according to the implemented matching criteria.
Non-functional Requirements	NFR03 Performance — matches generated within a reasonable period; NFR04 Location accuracy — accurate route/location comparison; NFR05 Scalability — matching supports increasing rides/requests.

Requirement ID	RQ06
Requirement Description	**Ride Request Management** — Drivers shall be able to respond to passenger requests. The system shall allow drivers to view requests associated with their ride offers and accept or decline individual requests. The corresponding request status shall be updated and stored.
Non-functional Requirements	NFR02 Reliability — status updates stored consistently; NFR06 Usability — drivers can respond with minimal difficulty.

Requirement ID	RQ07
Requirement Description	**Ride Status Management** — Users shall be able to know the status of a ride or request. The system shall maintain and update ride and request statuses, including relevant states such as requested, accepted, completed and cancelled. Updated statuses shall be available to the relevant users.
Non-functional Requirements	NFR02 Reliability — statuses updated without data loss or inconsistency.

Requirement ID	RQ08
Requirement Description	**Ride History** — Users shall be able to view their previous rides and requests. The system shall store records of relevant ride offers, ride requests and completed rides and shall allow authenticated users to retrieve and view their own ride history.
Non-functional Requirements	NFR02 Reliability — historical records retained accurately.

Requirement ID	RQ09
Requirement Description	**Interactive Map** — Users shall be able to view and select locations associated with their journeys. The system shall provide an interactive map through which users can select or view relevant origin and destination locations. The selected location information shall be associated with the relevant ride offer or request.
Non-functional Requirements	NFR04 Location accuracy — accurate locations/routes from mapping services; NFR06 Usability — simple location selection.

Requirement ID	RQ10
Requirement Description	**Notifications** — Users shall receive notifications about relevant ride events. The system should notify relevant users when events occur, including new ride requests, accepted or declined requests and relevant ride-status changes.
Non-functional Requirements	NFR03 Performance — timely delivery; NFR07 Availability — notifications delivered while core services are accessible.

Requirement ID	RQ11
Requirement Description	**Ride Search and Filtering** — Passengers shall be able to search for available rides that meet their travel requirements. The system shall allow passengers to search for available ride offers and apply relevant criteria, such as destination, departure time and available seats, to narrow the displayed results.
Non-functional Requirements	NFR03 Performance — search results returned promptly; NFR06 Usability — filtering with minimal difficulty.

Requirement ID	RQ12
Requirement Description	**Data Storage and Retrieval** — The system shall retain the information required to operate the ride-sharing service. The system should store and retrieve user profiles, ride offers, ride requests, ride statuses, seating information and ride histories using the project's database.
Non-functional Requirements	NFR02 Reliability — data stored without loss or inconsistency; NFR05 Scalability — database supports growing data volume.

Requirement ID	RQ13
Requirement Description	**Access Control** — Users shall only access functionality and information appropriate to their account. The system shall restrict protected functionality and user-specific information according to the authenticated user's account and role.
Non-functional Requirements	NFR01 Security — enforced through authentication and access controls.

Requirement ID	RQ14
Requirement Description	**Mobile Application Support** — Users shall be able to access the ride-sharing system through supported mobile devices. The system shall provide its core ride-sharing functionality through the developed mobile application on the supported Android and iOS platforms.
Non-functional Requirements	NFR06 Usability — consistent, easy-to-use mobile interface; NFR07 Availability — core functionality accessible on supported platforms.

Requirement ID	RQ15
Requirement Description	**Ride Matching Results** — Users shall receive the results of the ride-matching process. The system shall process submitted driver and passenger journey information and return potential compatible ride matches based on the implemented matching criteria.
Non-functional Requirements	NFR03 Performance — results generated within a reasonable period; NFR04 Location accuracy — accurate matches based on location/route data.

Requirement ID	RQ16
Requirement Description	**Core Task Completion** — Users shall be able to complete the core tasks required to use the ride-sharing system. The system should provide interfaces that allow drivers and passengers to complete the core tasks defined. The usability of these interfaces shall be evaluated using the System Usability Scale (SUS).
Non-functional Requirements	NFR06 Usability — evaluated via the System Usability Scale (SUS).